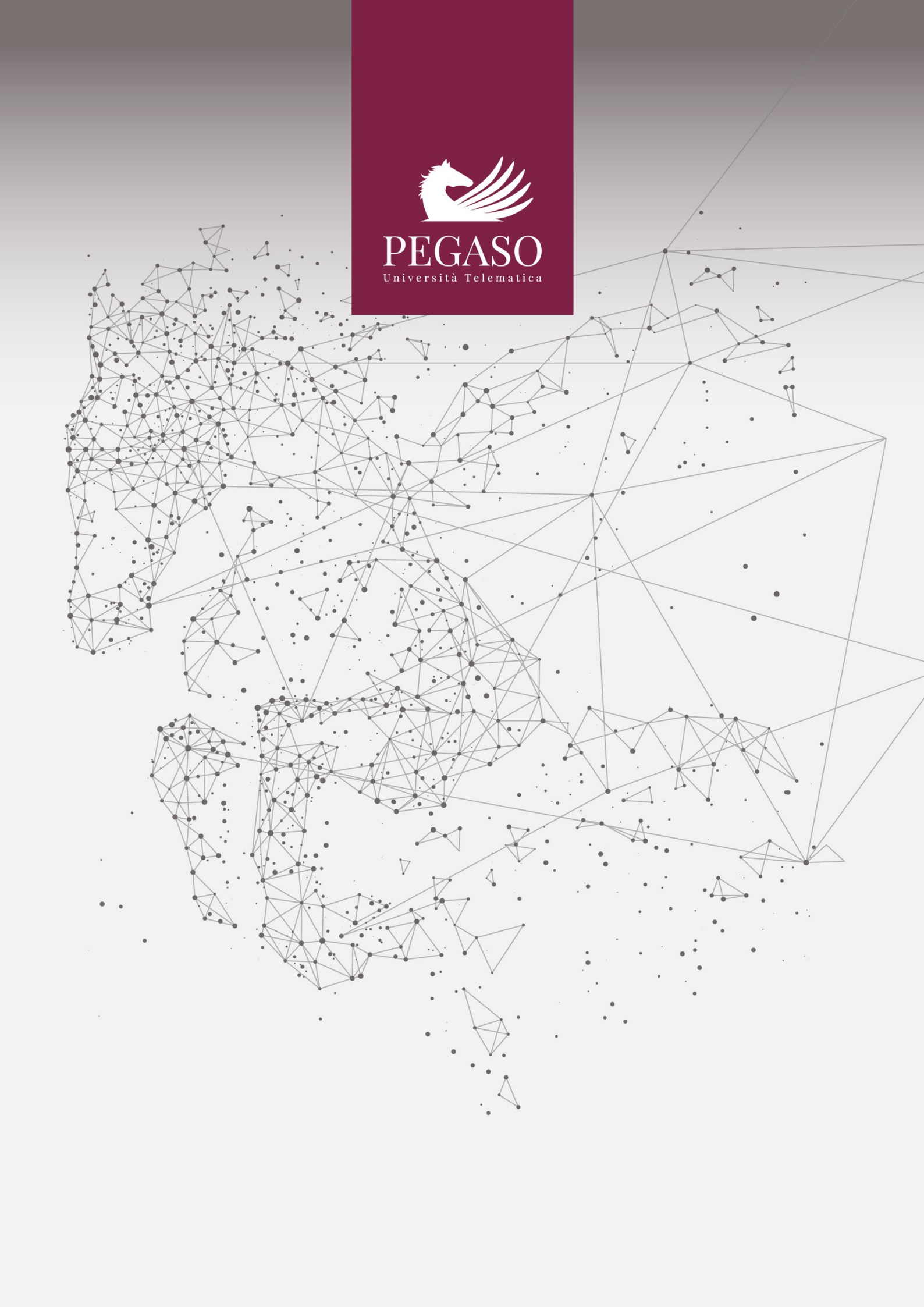

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. WIDGET TKINTER....4
3. LA WRAPPER CLASS LABELINPUT....11
4. OVERRIDING DEI METODI DELLA CLASSE LABELINPUT....14
BIBLIOGRAFIA....16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano i widget Tkinter di maggiore utilizzo nella composizione di un form: Entry, Spinbox, Combobox, Checkbutton, Text e Button. Per ciascuno dei widget si illustrano i parametri da passare al costruttore, un codice di esempio di utilizzo del widget e la resa grafica dell'esempio stesso. Si illustrano, inoltre, la classe wrapper LabelInput e l'overriding dei metodi grid, get e set di tale classe al fine di semplificare l'implementazione del form nella fase di definizione e composizione dei widget.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

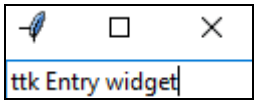
2. Widget Tkinter

A seguito della progettazione del *form* per l'inserimento dei dati, è possibile procedere scrivendo il codice per la sua implementazione. Ad ogni modo, prima della scrittura dell'intero *form* si analizzano i codici necessari per l'utilizzo dei diversi widget messi a disposizione da Tkinter, tra cui i più diffusi risultano essere:

1. *Entry;*
2. *Spinbox;*
3. *Combobox;*
4. *Checkbutton;*
5. *Text;*
6. *Button.*

2.1 Widget ttk.Entry

Il ***widget ttk.Entry*** è un normale rettangolo di inserimento di testo su una sola linea come mostrato nella figura seguente.



È possibile creare un widget ttk.Entry attraverso il seguente codice:

```
my_entry = ttk.Entry(
    parent,
    textvariable=my_text_var
)
```

dove:

- ***parent*** è l'argomento che imposta il *widget parent* (gerarchicamente superiore) che conterrà il widget Entry;
- ***textvariable*** è un oggetto di tipo *StringVar* di Tkinter il cui valore sarà collegato a questo widget di input;
- ***show*** (opzionale) è l'argomento che determina quale carattere sarà visualizzato quando si inserisce il testo nel *widget Entry*. Di default è il carattere inserito, ma in casi speciali come per le **password** potrebbe essere sostituito dall'asterisco come riportato nella figura seguente.

PEGASO Università Telematica

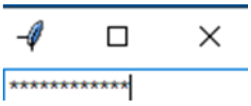
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tra tutti gli argomenti visti, anche l'uso dell'argomento *textvariable* è opzionale. Senza tale argomento, infatti, è possibile estrarre il valore all'interno del widget entry utilizzando il metodo ***get***().

Ad ogni modo legare un oggetto di tipo *StringVar* al widget di input presenta alcuni **vantaggi**:

1. non è

```
my_entry = ttk.Entry(
    parent,
    textvariable=my_text_var,
    show='*'
)
```



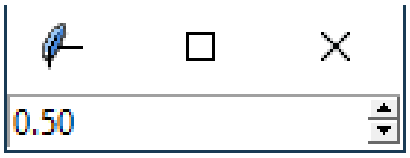
necessario passare o mantenere un riferimento al widget;

2. è più semplice organizzare il software in moduli separati;
3. le variazioni al valore dell'input sono automaticamente propagate all'oggetto *StringVar* e viceversa.

Il codice seguente permette di visualizzare un **widget Entry** a schermo. Nella figura seguente a destra si riporta il risultato dell'elaborazione del codice a sinistra.

2.2 Widget ttk.Spinbox

Il ***widget ttk.Spinbox*** aggiunge *due pulsanti* (uno per l'incremento e uno per il decremento) ad un normale *widget Entry* rendendolo adatto a **dati numerici**. Nella figura seguente si riporta un esempio di widget ttk.Spinbox.



Il codice per creare un widget Spinbox è il seguente:

```
my_spinbox = tk.Spinbox(
    parent,
    from_=0.5,
    to=52.0,
    increment=.01,
    textvariable=my_double_var
)
```

dove:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- ***parent*** è l'argomento che imposta il *widget parent* (gerarchicamente superiore) che conterrà il widget Spinbox;
- ***from_*** è l'argomento che determina il **minimo valore esprimibile dal widget**. Il trattino basso è necessario perché *from* è una *keyword Python;*
- ***to*** è l'argomento che determina il massimo valore esprimibile dal widget;
- ***increment*** è l'argomento che rappresenta la quantità di incremento/decremento;
- ***textvariable*** è un oggetto *StringVar* di Tkinter il cui valore sarà collegato a questo widget di input.

Sia *from_* che *to* sono obbligatori nel momento in cui si usa uno dei due. Specificare, ad esempio, solo il limite inferiore può causare, infatti, comportamenti non previsti.

Il **widget Spinbox** non è utilizzato solo per i numeri (anche se primariamente viene utilizzato per essi). È possibile, infatti, utilizzare una *lista di stringhe* che possono essere selezionate attraverso le freccette sul widget. Proprio perché con il widget Spinbox possono essere utilizzati sia stringhe che numeri, l'argomento *textvariable* può accettare dati di tipo *StringVar, IntVar* o *DoubleVar.*

Il codice seguente permette di visualizzare un **widget Spinbox** a schermo. Nella figura seguente a destra si riporta il risultato dell'elaborazione del codice a sinistra.

```
import tkinter as tk
from tkinter import ttk

parent = tk.Tk()
my_double_var = tk.DoubleVar()

my_spinbox = tk.Spinbox(
    parent,
    from_=0.5,
    to=52.0,
    increment=.01,
    textvariable=my_double_var
)
my_spinbox.pack()
parent.mainloop()
```

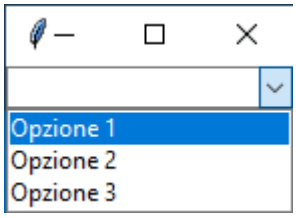


2.3 Widget ttk.Combobox

Il **widget ttk.Combobox** è un *widget Entry* che aggiunge un ***menu di selezione drop-down.***

Per popolare il menu drop-down è necessario passare all'argomento ***values*** una lista di stringhe, tra cui l'utente può scegliere. Nella figura seguente si riporta un esempio di widget Combobox.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Il codice per creare un widget Combobox è il seguente:

```
combobox = ttk.Combobox(
    parent,
    textvariable=my_string_var,
    values=["Opzione 1", "Opzione 2", "Opzione 3"]
)
```

dove:

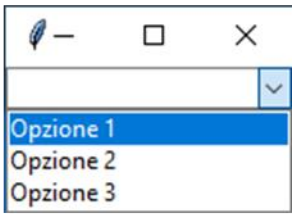
- ***parent*** è l'argomento che imposta il widget parent (gerarchicamente superiore) che conterrà il *widget Combobox;*
- ***textvariable*** è un oggetto di tipo *StringVar* di Tkinter il cui valore sarà collegato a questo widget di input;
- ***values***, come anticipato, è una lista di stringhe che rappresentano le opzioni, tra cui l'utente può scegliere.

Il codice seguente permette di visualizzare un **widget Combobox** a schermo. Nella figura seguente a destra si riporta il risultato dell'elaborazione del codice a sinistra.

```
import tkinter as tk
from tkinter import ttk

parent = tk.Tk()
my_string_var = tk.StringVar()

combobox = ttk.Combobox(
    parent,
    textvariable=my_string_var,
    values=["Opzione 1", "Opzione 2", "Opzione 3"]
)
combobox.pack()
parent.mainloop()
```



PEGASO
Università Telematica

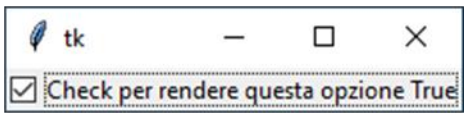
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import tkinter as tk
from tkinter import ttk

parent = tk.Tk()
my_boolean_var = tk.BooleanVar()

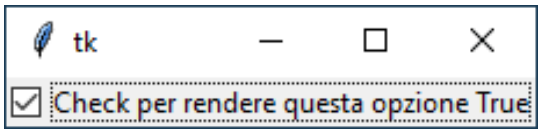
my_checkbutton = ttk.Checkbutton(
    text="Check per rendere questa opzione True",
    variable=my_boolean_var
)

my_checkbutton.pack()
parent.mainloop()
```



2.4 Widget ttk.Checkbutton

Il **widget ttk.Checkbutton** è un ***checkbox etichettato*** per l'inserimento di *dati booleani.* A differenza dello *Spinbox* e del *Combobox* non deriva dal *widget Entry* e i suoi argomenti sono diversi. Nella figura seguente si riporta un esempio di widget Checkbutton.



Il codice per creare un *widget Checkbutton* è il seguente:

```
my_checkbutton = ttk.Checkbutton(
    text="Check per rendere questa opzione True",
    variable=my_boolean_var
)
```

dove:

- ***text*** è l'argomento che imposta l'etichetta del widget;
- **variable** è l'argomento di tipo BooleanVar a cui lo stato booleano è collegato;
- ***textvariable*** a differenza del widget Entry questo argomento può essere usato per collegare una variabile al testo dell'etichetta del widget. Con questo widget, *textvariable* non viene utilizzato molto di frequente anche se è utile sapere della sua esistenza.

Il codice seguente permette di visualizzare un **widget Checkbutton** a schermo. Nella figura seguente a destra si riporta il risultato dell'elaborazione del codice a sinistra.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2.5 Widget tk.Text

Il **widget tk.Text** è un widget che è molto di più di un *widget Entry multilinea.* Esso fornisce, infatti, un ***sistema di tag*** molto potente che permette di implementare testo multicolore, link cliccabili e molto altro. A differenza degli altri widget ***non può essere collegato ad una StringVar*** Tkinte. Ne segue che l'impostazione il recupero del testo contenuto dovrà essere eseguito attraverso i metodi *get*(), *insert*() e *delete*().

Quando si legge o si modifica il testo con i metodi *get*(), *insert*() e *delete*() è necessario passare anche i valori ***index*** per selezionare il *carattere* (un valore di index) o un *intervallo di caratteri* (*due valori di index*) su cui operare. I valori di **index** vanno passati come ***stringhe*** secondo uno dei seguenti formati:

1. il ***numero della linea*** e il ***numero del carattere*** separati da un punto. Le linee sono numerate da 1, mentre i caratteri da 0. Il primo carattere della prima linea è, dunque, 1.0;
2. la stringa **end** o la costante ***END*** di Tkinter indica la fine del campo del widget Text;
3. un ***indice numerico*** più una delle seguenti parole ***linestart, lineend, wordstart, wordend*** indica l'inizio o la fine della linea o della parola relativa all'indice numerico. Ad esempio, *6.2 wordstart* corrisponde all'inizio della parola contenente il terzo carattere della linea 6, mentre *2.0 lineend* corrisponde alla fine della linea due.

Qualsiasi dei formati precedenti insieme a un ***operatore di somma/differenza*** e un ***numero di caratteri o linee***. Ad esempio, *2.5 wordend – 1 chars* corrisponde al carattere prima della fine della parola contenente il sesto carattere della linea 2.

Il codice per creare un **widget Text** è il seguente:

```
mytext = tk.Text(parent)
```

dove:

- ***parent*** è l'argomento che imposta il widget *parent* (gerarchicamente superiore) che conterrà il widget Text.

Il seguente codice mostra le basi dell'uso del **widget Text**. Nella figura seguente a destra si riporta il risultato dell'elaborazione del codice a sinistra.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2.6 Widget ttk.Button

Il **widget ttk.Button** è un ***pulsante cliccabile*** con il mouse o lo spazio della tastiera. Nella figura seguente si riporta un esempio di ttk.Button.



Il codice per creare un **widget Button** è il seguente:

```
my_button = ttk.Button(parent, textvariable=tvar, command=swaptext)
```

Come il widget Checkbutton, questo widget usa le opzioni di configurazioni *text* e *textvariable* per controllare l'etichetta sul pulsante. L'argomento ***command***, invece, specifica una funzione Python (funzione di callback) da invocare quando il pulsante viene cliccato.

```
import tkinter as tk


pare import tkinter as tk
# cr from tkinter import ttk

myte     parent = tk.Tk()

# in
myte tvar = tk.StringVar()
     def swaptext():
# in       tvar.set('Mondo' if tvar.get() == 'Ciao' else 'Ciao')
myte
     my_button = ttk.Button(parent, textvariable=tvar, command=swaptext)
# re
myte my_button.pack()

# el
# l' parent.mainloop()
# quindi per eliminare l ultimo carattere
# è necessario tornare indietro di 2 caratteri
mytext.delete('end - 2 chars')


mytext.pack()
parent.mainloop()
```



Il seguente codice mostra un esempio di uso del ***widget Button*** per modificare l'etichetta del pulsante ad ogni click del mouse. Nella figura seguente a destra si riporta il risultato dell'elaborazione del codice a sinistra, rispettivamente dopo 0, 1, 2 click del mouse sul pulsante.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. La wrapper class LabelInput

Una volta noto il codice necessario per ciascun widget, è possibile istanziare ciascun widget e inserirlo nel *form* con una etichetta ad esso associata. In una piccola applicazione è possibile creare separatamente *l'etichetta* e *l'input* e, successivamente, aggiungere ciascuno di essi al frame *parent* come nel seguente caso:

```
from tkinter import Frame, Label, Entry

form = Frame()
label = Label(form, text='Name')
name_input = Entry(form)
label.grid(row=0, column=0)
name_input.grid(row=1, column=0)

form.pack()
form.mainloop()
```

Il codice appena visto funziona molto bene, ma crea anche una **maggiore quantità di codice** (ripetitivo) che è più difficile da manutenere. Visto che i **widget Label** e di **input** (es. *Entry*) vengono utilizzati insieme, è utile creare una *classe wrapper contenente entrambi i widget.*

Definiamo, quindi, una **classe wrapper LabelInput.**

Il codice iniziale della **classe wrapper LabelInput** è il seguente:

```
import tkinter as tk
from tkinter import ttk


class LabelInput(tk.Frame):
    #Un widget che contiene un etichetta e un input insieme

    def __init__(self, parent, label='', input_class=ttk.Entry,
                 input_var=None, input_args=None, label_args=None,
                 **kwargs):
        super().__init__(parent, **kwargs)
        input_args = input_args or {}
        label_args = label_args or {}
        self.variable = input_var
```

In primo luogo, si fa derivare la ***classe LabelInput*** dalla classe ***Tkinter.Frame*** come si può osservare nella porzione seguente del codice:

```
class LabelInput(tk.Frame):
    #Un widget che contiene un etichetta e un input insieme
```

Il costruttore della nuova classe riceve i seguenti argomenti:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    def __init__(self, parent, label='', input_class=ttk.Entry,
                 input_var=None, input_args=None, label_args=None,
                 **kwargs):
```

dove:

- ***parent*** è il riferimento al widget parent che conterrà il *widget LabelInput*;
- ***label*** è il testo delle etichette del nuovo widget;
- ***input_class*** è la classe del widget che si vuole creare. Deve essere un ***oggetto input***, *non una stringa*. Se lasciato vuoto, il default del tipo di input è impostato come *widget ttk.Entry;*
- ***input_var*** è una variabile Tkinter a cui assegnare l'input. *È opzionale* in quanto alcuni widget di input non usano le variabili;
- ***input_args*** è un *dizionario opzionale* per argomenti aggiuntivi per il costruttore del ***widget di input***;
- ***label_args*** è un *dizionario opzionale* per argomenti aggiuntivi per il costruttore del ***widget Label***;
- *****kwargs*** serve per raccogliere argomenti keyword aggiuntivi da passare al *costruttore del Frame*.

La prima istruzione all'interno del costruttore sarà la chiamata al costruttore della superclasse frame ***super().__init__()*** passando l'argomento ***parent*** e gli argomenti keyword aggiuntivi *****kwargs***. Successivamente, si verifica che gli argomenti ***input_args*** e ***label_args*** siano dei **dizionari**. Si salva, inoltre, il riferimento alla variabile di input come ***self.variable***. La porzione seguente del codice mostra le prime quattro istruzioni del costruttore:

```
        super().__init__(parent, **kwargs)
        input_args = input_args or {}
        label_args = label_args or {}
        self.variable = input_var
```

La **classe LabelInput** dovrà gestire qualsiasi tipo di widget di input. Ad ogni modo tra i costruttori (e il funzionamento) dei diversi widget ci sono delle piccole differenze come l'utilizzo dell'argomento *textvariable* per i ***widget Combobox*** e ***Checkbutton***.

Per il momento, quindi, differenzieremo il comportamento dei ***widget con pulsante*** (Button e Checkbutton) da ***quelli con i testi etichettati.*** Per tale motivo, al ***costruttore della classe LabelInput*** aggiungeremo il seguente codice:

```
if input_class in (ttk.Checkbutton, ttk.Button, ttk.Radiobutton):
    input_args["text"] = label
    input_args["variable"] = input_var
else:
    self.label = ttk.Label(self, text=label, **label_args)
    self.label.grid(row=0, column=0, sticky=(tk.W + tk.E))
    input_args["textvariable"] = input_var
```

Per i *widget con pulsante*, invece di aggiungere un'etichetta, imposteremo l'argomento ***text*.** Tutti i widget con pulsante usano, infatti, l'argomento *text* per *aggiungere un'etichetta al widget*. Inoltre, sempre

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

per i *widget con pulsante*, invece di assegnare la variabile a *textvariable*, si imposta l'argomento ***variable*** a *input_var*.

Per gli *altri tipi di widget di input,* impostiamo *textvariable* e creiamo un **widget Label** aggiungendolo alla prima colonna della prima riga della classe LabelInput con geometria a griglia.

Dopo aver impostato i parametri creiamo la *classe di input* come segue (di default ***input class sarà ttk.Entry***):

```
self.input = input_class(self, **input_args)
self.input.grid(row=1, column=0, sticky=(tk.W + tk.E))
self.columnconfigure(0, weight=1)
```

Nel codice precedente, si chiama, quindi, il costruttore della classe di input desiderata *input_class* passandogli il dizionario *input_args* e si aggiunge il widget di input alla griglia alla riga 1. Si configura, poi il **layout grid** per espandere l'unica colonna dell'intero widget attraverso l'istruzione:

self.columnconfigure(0, weight=1)

Il codice finale del ***costruttore di LabelInput*** sarà il seguente:

```
def __init__(self, parent, label='', input_class=ttk.Entry,
             input_var=None, input_args=None, label_args=None,
             **kwargs):
    super().__init__(parent, **kwargs)
    input_args = input_args or {}
    label_args = label_args or {}
    self.variable = input_var

    if input_class in (ttk.Checkbutton, ttk.Button, ttk.Radiobutton):
        input_args["text"] = label
        input_args["variable"] = input_var
    else:
        self.label = ttk.Label(self, text=label, **label_args)
        self.label.grid(row=0, column=0, sticky=(tk.W + tk.E))
        input_args["textvariable"] = input_var

    self.input = input_class(self, **input_args)
    self.input.grid(row=1, column=0, sticky=(tk.W + tk.E))
    self.columnconfigure(0, weight=1)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Overriding dei metodi della classe LabelInput

Visto che tutti gli **oggetti (widget) LabelInput** dovranno riempire l'intera cella della griglia in cui sono posti, si procede con ***l'overriding del metodo grid***() (tra i metodi della classe LabelInput) con il seguente codice:

```
def grid(self, sticky=(tk.E + tk.W), **kwargs):
    super().grid(sticky=sticky, **kwargs)
```

In questo modo, si eviterà di aggiungere *sticky=(tk.E + tk.W)* a ciascuna chiamata *LabelInput.grid().*

Tutti widget di input hanno, inoltre, tra i metodi, il metodo **get**() che restituisce il loro valore corrente. Per ridurre la ridondanza del codice in fase di utilizzo, si procederà con ***l'overriding anche del metodo get***() che restituirà come valore del widget LabelInput il *valore del widget di input* o *quello della variabile*. L'overriding del metodo *get*() avviene con il seguente codice:

```
def get(self):
    try:
        if self.variable:
            return self.variable.get()
        elif type(self.input) == tk.Text:
            return self.input.get('1.0', tk.END)
        else:
            return self.input.get()
    except (TypeError, tk.TclError):
        return ''
```

Nel codice precedente, viene utilizzato un *blocco try-except* perché le variabili Tkinter lanciano un'eccezione dopo la chiamata a **get**() in alcune circostanze, ad esempio, se un campo numerico è vuoto. Nel caso di eccezione verrà restituita una stringa vuota. Anche i **widget tk.Text** dovranno essere gestiti in modo diverso poiché richiedono degli indici (nel nostro caso da 1.0 a tk.END) per recuperare tutto il testo contenuto nel widget.

In modo complementare all'overriding del metodo get(), si procede ***all'overriding del metodo set***() nel seguente modo:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def set(self, value, *args, **kwargs):
    if type(self.variable) == tk.BooleanVar:
            self.variable.set(bool(value))
    elif self.variable:
            self.variable.set(value, *args, **kwargs)
    elif type(self.input).__name__.endswith('button'):
        if value:
            self.input.select()
        else:
            self.input.deselect()
    elif type(self.input) == tk.Text:
        self.input.delete('1.0', tk.END)
        self.input.insert('1.0', value)
    else:
        self.input.delete(0, tk.END)
        self.input.insert(0, value)
```

Nel caso di una ***variabile di classe booleanVar***, si esegue il *casting a bool* e si richiama il metodo **set**() della variabile stessa. Per gli altri tipi di variabili si passa ***value*** direttamente al loro metodo **set**(). Nel caso in cui si sia in presenza di un **widget con pulsante**, si utilizzano i metodi *select*() e *deselect*() rispettivamente per selezionare e deselezionare il pulsante in base al valore di verità della variabile. Nel caso in cui sia un ***widget tk.Text*** è possibile utilizzare i metodi *insert*() e *delete*(), altrimenti si richiamano i metodi *delete*() e *insert*() dei widget di input che sono invocabili con le classi *Entry*, *Spinbox* e *Combobox.* Si noti che è necessario differenziare questo caso da quello del ***widget di tipo tk.Text*** perché l'indicizzazione funziona in modo differente.

Ovviamente la ***classe LabelInput*** per come è stata scritta non gestisce tutti i possibili widget di input, ma copre quelli che verranno utilizzati per il form.

La costruzione della ***classe LabelInput*** ha richiesto molto lavoro, ma grazie ad esso, l'implementazione del form sarà molto più semplice.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Moore A. D. (2018). Python GUI Programming with Tkinter. Packt

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).